

Chat-like Asserts Prediction with the Support of Large Language Model

HAN WANG, Faculty of Information Technology, Monash University, Australia

HAN HU, Faculty of Information Technology, Monash University, Australia

CHUNYANG CHEN, Technical University of Munich & Monash University, Germany

BURAK TURHAN, M3S Research Unit, Faculty of ITEE, University of Oulu, Finland

Unit testing is an essential component of software testing, with the assert statements playing an important role in determining whether the tested function operates as expected. Although research has explored automated test case generation, generating meaningful assert statements remains an ongoing challenge. While several studies have investigated assert statement generation in Java, limited work addresses this task in popular dynamically-typed programming languages like Python. In this paper, we introduce Chat-like execution-based Asserts Prediction (CLAP), a novel Large Language Model-based approach for generating meaningful assert statements for Python projects. CLAP utilizes the persona, Chain-of-Thought, and one-shot learning techniques in the prompt design, and conducts rounds of communication with LLM and Python interpreter to generate meaningful assert statements. We also present a Python assert statement dataset mined from GitHub. Our evaluation demonstrates that CLAP achieves 64.7% accuracy for single assert statement generation and 62% for overall assert statement generation, outperforming the existing approaches. We also analyze the mismatched assert statements, which may still share the same functionality and discuss the potential help CLAP could offer to the automated Python unit test generation. The findings indicate that CLAP has the potential to benefit the SE community through more practical usage scenarios.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Unit Testing, LLM, Asserts Prediction

1 INTRODUCTION

Unit testing plays an important role in software testing, as it can detect and diagnose bugs in the existing system by testing small units at the early stage [28, 68]. A unit test case normally consists of test inputs and test oracles (assert statements) [82]. The assertions are crucial in the unit testing process, as they compare the actual and expected outputs to determine whether a tested function performs as intended. High-quality assert statements enhance the maintainability of unit tests, help better identify bugs, and assist future developers in understanding the purpose and output of the code. In addition, if a test case fails, the assert statements serve as a valuable reference for identifying and resolving faults.

Developing unit tests for a project can be a demanding and time-consuming task for developers [46, 78]. In response to this challenge, automated unit test generation has emerged as a well-established field of research. Studies have concentrated on automating unit test generation using rule-based or search-based algorithms for both Java and Python projects [32, 45, 47, 64]. However, while search-based testing approaches can facilitate the automation of test input generation and improve test coverage, generating complete and meaningful assert statements that meet the actual needs of developers remains a challenge [75]. To be specific, these approaches either capture and assert the return values of non-void-return methods or introduce mutants into the

Authors' addresses: Han Wang, han.wang@monash.edu, Faculty of Information Technology, Monash University, Australia; Han Hu, han.hu@monash.edu, Faculty of Information Technology, Monash University, Australia; Chunyang Chen, chunyang.chen@tum.de, Technical University of Munich & Monash University, Germany; Burak Turhan, burak.turhan@oulu.fi, M3S Research Unit, Faculty of ITEE, University of Oulu, Finland.

2024. XXXX-XXXX/2024/8-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

software and attempt to generate assert statements able to kill these mutants. Interviews [21] with developers have revealed that the automatically generated assertions are often poor and simple, resulting in their inability to detect real bugs, and the lack of meaningfulness in these assertions leads developers to prefer handwritten test cases over the generated ones, ultimately undermining the purpose of automated testing [75].

Therefore, recent works have started to explore automatically generating meaningful assert statements in Java given the input of the test input and its focal method (method to be tested) [30, 50, 52, 69, 75, 82]. However, for dynamically typed programming languages such as Python, though it has become the most popular programming language [15], no previous research has investigated the generation of meaningful assert statements. There is a demand for more advanced and effective tools to assist developers in this area and enhance the overall code quality. Unlike Java, Python is dynamically typed, features flexible syntax, and employs high-level abstractions, making it more challenging to accurately capture the expected behaviour or values of the focal method. Directly adopting current approaches to assert statement generation in Python has limitations. First, these deep learning (DL) or information retrieval (IR) based methods rely on a training set, and performance varies with the set's size and relevance. Unfortunately, Python lacks a standardised dataset specifically for assert generation, complicating the adoption of these models. Python's diverse testing practices and dynamic typing system, is challenging for automated test identification and assert extraction, further compounded by a less mature testing tools ecosystem that limits the availability of training data for such methods. Second, we found that 71.8% of the test cases we crawled from open-source Python projects contain more than one assert statement in a test case, which is not supported by current approaches designed only for a single assert statement. Generating multiple assertions using the current single assert statement can be challenging as identified in previous work [75]. These methods typically treat each assertion independently, but fail to consider the connections/dependencies of the asserts in the same test cases. These limitations highlight the need for new approaches to generate multiple meaningful assert statements for Python unit testing.

In recent years, Large Language Models (LLM), such as GPT [11] and LLAMA [66, 67], have emerged as pre-trained neural network models capable of achieving impressive results in various tasks such as code completion [35, 42, 48, 53], documentation and comments generation [34, 38], repair programs and fix bugs [25, 52, 57]. Leveraging LLMs' pre-trained nature and their proficiency in understanding code context, we use them to automate the assert generation in Python. Notably, GPT models perform well at Python code generation compared to other languages, likely due to their pre-release training set distribution [11, 26].

In this paper, we propose CLAP (Chat-like LLM-based Asserts Prediction), a novel LLM-based approach for generating meaningful assert statements for Python projects. We adopt state-of-art prompt design techniques, such as one-shot learning and Chain-of-Thought (CoT), into the assert generation prompts. To include more context of the given code, we design three prompt templates to communicate with the LLM as well as support the multi asserts generation given one test case. We execute the initial round of predictions and relay any error messages received from the interpreter back to the LLM, prompting it to correct the mistakes and refine its predictions. We also mine GitHub and release an open-source Python assert statement dataset. Lastly, we implement the approach with OpenAI LLMs and evaluate it with the dataset.

We evaluate the effectiveness of our prompt design and compare CLAP performance with state-of-the-art approaches [52, 74, 75]. While the prompt design demonstrates strong performance with the two OpenAI LLMs, it achieves better results on the *text-davinci-003* [11]. The prompt techniques we integrate into CLAP can help the LLM generate more accurate assertions. In our evaluation, CLAP attains 64.7% accuracy for single assert statement generation, and achieves 62% accuracy in

generating all assert statements that no other approaches have explored. We collect the generated assertions that can improve the test code quality or fit more to the test purpose and submit pull requests to the tested projects, and 7 out of 9 pull requests has been approved.

In summary, the contribution of this work is:

- The study specifically focuses on assert generation in Python, encompassing multi-assert generation and the utilization of various testing libraries.
- CLAP, a system that integrates rounds of CoT and feedback prompts to interact with the LLM, effectively addressing the assert generation task.
- We make both our tool, CLAP, and our Python assert dataset publicly available for the benefit of the research community [6].

2 BACKGROUND

2.1 Unit Testing & Assert Statements

A unit test is a small and executable piece of code that exercises the functionality of a unit under test [21]. Unit testing plays an important role in test-driven development (TDD) as it can better help with fault detection [65], contribute to developer's productivity [33], and can also serve as documentation and specification [63]. The existence of unit tests ensures the correctness, reliability, and maintainability of the units within the code [36, 81].

To implement unit tests, developers use specific conventions and frameworks that define how test methods are structured and recognized within different programming environments. In Java projects, test methods start with `@Test` annotation (inherent to the JUnit framework) [75]. However, in Python, projects do not have a universal annotation for indicating test methods. Instead, they often rely on naming conventions and specific testing frameworks such as `unittest` [16]. In Python, test methods are generally prefixed with `test` [68] and contained modules following the pattern `test_module_name.py` or `module_name_test.py`.

Test oracles are important parts of the test methods, normally appearing as assert statements. They validate whether the given conditions are met during the execution. In Python, there are two common ways to write assert statements. The first method involves utilising the assert keywords provided by Python, which allows for limited but straightforward checks of conditions within the code. The second approach entails leveraging other testing libraries, such as `unittest` [16], `scikit-learn` [14] and `tf.test` [10], which offer over 70 specialized assert statements and additional features for a wide range of testing scenarios. For example, the `assertRaisesRegex` checks if a specific exception is raised during the execution and if the exception message matches the regular expression pattern. Therefore, the assert statements in Python vary depending on the specific requirements and preferences of developers.

2.2 Automatic Assert Generation

Automatic assert generation is a critical challenge in software testing. It aims to automate the creation of assert statements within unit tests. The challenge in automatic assert generation lies in effectively predicting assert statements that accurately reflect the intended functionality of the application.

The pioneering work in this area was introduced by Watson et al. [75] in 2020. They developed a Deep Learning (DL) approach using a Neural Machine Translation (NMT) model, named ATLAS, to generate assert statements specifically for Java. They proposed the Test-Assert Pairs (TAPs) dataset extracted from Java unit test method. This dataset paired unit tests with their corresponding focal methods, providing a structured way to train and evaluate the model. Since then, all works have followed this structure, generating assert statements based on the given rest of the unit test and its

focal method. Subsequent developments [49, 50, 52, 69, 82] have enhanced the assert generation in Java with mixed methods, including information retrieval and the tuning of transformer and large language models. Beyond Java, developments in other languages like JavaScript have also progressed; for instance, Zamprogno et al. [83] introduced AutoAssert, which incorporates a human-in-the-loop mechanism. The advancements highlight the increasing interest in automatic assert generation, aimed at reducing manual efforts.

2.3 Large Language Model & Prompt Design

Large Language Models (LLMs) like GPT, with over 175 billion parameters, excel in diverse tasks such as summarizing and code completion. LLMs are Trained on extensive datasets, including various open-source projects, and built on the Transformer architecture [71]. They utilise mechanisms like self-attention to comprehend prompts and capture contextual relationships between words from different distances. To control the output of LLMs, prompts are fed into the models. Prompt design influences the LLM’s ability to capture the requirements and produce specific and coherent results. The concept of prompt engineering [43] has been proposed to describe the iterative process of crafting and refining prompts to optimize the performance of LLMs in generating desired outputs. The process may include incorporating natural language descriptions, such as “*The aim of this task is to xxx*”, or give the LLMs one or a few samples (one-shot or few-shots learning) to better understand the task and guide the model in formatting the output content.

3 METHODOLOGY

In this section, we introduce CLAP, a system that engages the LLM as a role of an experienced unit test expert, and constructs prompts to initiate multiple conversation with the LLM to automatically generate the assert statements based on a given Python unit test case. Fig 1 illustrates the system overview of CLAP. Given a Python project, CLAP analyses the project and creates data entries that include the focal methods, test methods, and corresponding assert statements. Then, CLAP starts to communicate with the LLM, which involves three phases. First, we create a greeting prompt to define the task and provide one sample test case, offering LLM additional context (3.2). In the second phase, we incorporate the target test case into the query and wait for the initial round of assert predictions (3.3). In the third phase, we execute the predicted assert statements using a Python interpreter and, if any error messages arise, input these messages back into the LLM to request corrections to its predictions (3.4). By interacting with the LLM, CLAP collects the automatically generated assert statements and completes the unit test case.

3.1 Project Analysis

Our goal is to generate assert statements in Python projects. As of March 2023, Python has been acknowledged as the most popular programming language by the TIOBE Programming Community Index [15]; however, previous works have not focused on the assert generation area in Python. Therefore, CLAP begins by analysing the projects and preparing the data entries for the prompts construction. In Fig 2, we provide an example of how a data entry extracted from a project is presented. A data entry consists of three main parts, the focal method, unit test, and assert statements.

Within a given project, CLAP initially identifies all unit test files. This identification is based on the naming conventions of Python unit tests, as discussed in Section 2.1, checking for specific patterns in the file names. Upon locating the files, CLAP then examines their contents to verify the use of testing libraries and the completeness of the tests.

¹Code Sampled from: <https://github.com/scrapy/scrapy>

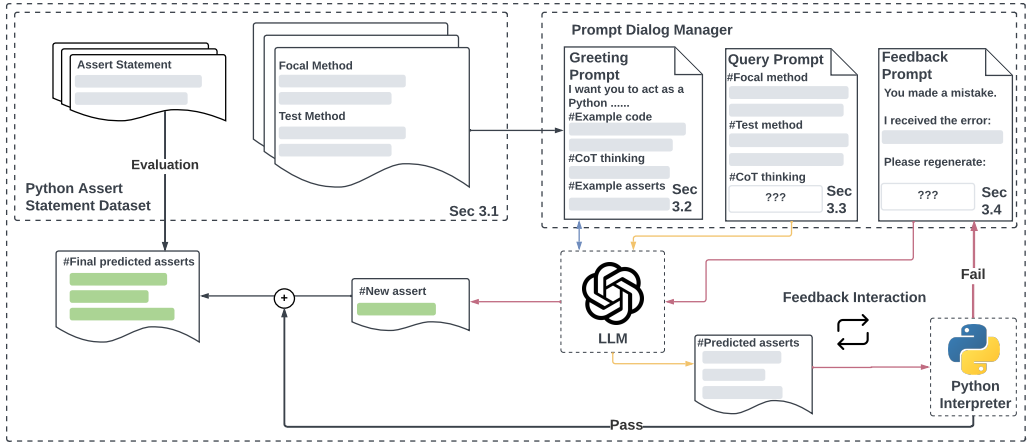


Fig. 1. The overview of our approach CLAP.

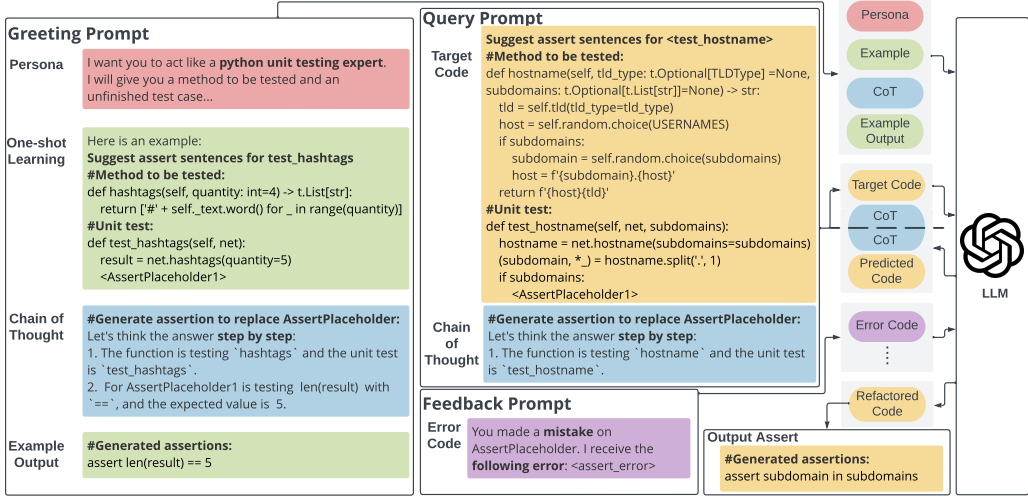
```
import tensorflow as tf
def test_conv2d(self):
    ...
    output = cl.conv2d(inputs, 'test_conv2d',
        filter_size, in_channels, out_channels, strides)
    output_shape = [2, 5, 5, 4]
    self.assertAllEqual(tf.shape(output), output_shape)
```

Fig. 2. Example code after the data extraction.¹

Next, CLAP starts to identify the focal method of the giving test method. Following the focal method extraction for Java projects [58, 75], we identify the last method call from the same library or project before the assert statements as the focal method. In some cases, the method calls are embedded inside the assert statements as parameters, for example, the *process_spider_input* in Fig 2. This approach might be considered controversial, as the parameter to be predicted is already present in the prompts. However, in real-world scenarios, developers typically have a clear understanding of the focal method. Consequently, we believe that incorporating the focal method information in the prompts closely aligns with developers' actual workflow and context, providing a more realistic and practical representation of the problem at hand. CLAP retained the original comments within both the focal methods and the unit tests, as these help the language model understand the functionality of the functions. Global variables defined in the same file were also included.

Finally, CLAP extracts the assert statements from the unit test and use `<AssertPlaceholder>` to replace the assert statements within the unit test code. To differentiate assert statements in the same test case, CLAP assigns incremental numbers to each of the statements. The detailed implementation is discussed in Section 4.1.

²Code sampled from: <https://github.com/lk-geimfari/mimesis>

Fig. 3. Prompt templates and sample responses.²

3.2 Greeting Prompt

The greeting prompt is the first prompt we sent to the LLM. As shown in Fig 3, it includes a persona message, a sample code for one-shot learning, a Chain of Thoughts (CoT) prompting, and the corresponding assert statement as the output.

Persona-based Prompting. The persona-based prompting approach assigns a persona or role related to the task given to the LLM [77]. By adopting a persona or role, the LLM can align its responses with the expertise, knowledge, and context associated with the given persona. This approach provides a clear framework and perspective for the LLM's response generation, ensuring that the output is more relevant and accurate for a specific task. In the official OpenAI documents, they also ask users to have a system message to help set the behaviour of the assistant [11]. In this study, we instruct the LLM to assume the role of a unit testing expert to assist in completing the unfinished test method by generating assert statements. The first sentence introduces the LLM to its assigned role. Then, inspired by a popular open-source repository that features a collection of valuable prompt examples [5], we include a task description paragraph to the prompt. The description details the input, steps to generate the assert statements, whether or not to use a certain testing framework, and the expected output, effectively guiding the LLM in producing the desired results.

One-shot Learning. The LLM, such as GPT-3, can achieve state-of-the-art performance on a wide variety of tasks using only few-shot prompts, in which examples of solved tasks (shots) are included in the prompt. Our approach adopts the one-shot learning prompt to generate assert statements for Python projects, enabling the LLM to learn from a single example and adapt its response generation accordingly. This method considers Python's unique language characteristics and assumes that the writing style remains consistent across the same project. Therefore, while retrieving this one sample, we only look for the test method that is within the same test class of the target test method as the sample. If there is only one test method in a class, we use the other test method within the same project as the sample input. When more than two test methods are present in a class, we compute the cosine similarity of the method names and choose the one with the highest score. After locating the sample code, we parse it into a format suitable for constructing the

Greeting Prompt, as illustrated in Fig 3, and generate the corresponding CoT process. We carried out a small-scale experiment showing that the performance of one-shot and few-shots prompting (6 shots) in our task does not show significant differences. In fact, one-shot prompting achieved higher accuracy (69.8% over 58.1%), likely due to less disruption in the prompts. Given the comparable performance and lower cost, we adopted one-shot prompting in CLAP.

Chain-of-thought Prompting. Recent research [39, 76, 85] in prompt design has demonstrated that for complex tasks requiring multi-step reasoning, incorporating a rationale or key phrase can significantly enhance the performance of LLMs. The approach, called Chain-of-thought (CoT) prompting, is effective in guiding LLMs toward desired outputs. In our work, we implement CoT prompting strategies by incorporating a guiding phrase followed by rational steps for generating assert statements. This approach aims to leverage the advantages of both strategies to enhance the LLM's ability to produce accurate results. To develop the manual instruction steps, we investigated the process of writing unit tests and assert statements [9, 22]. Additionally, we posed this question (i.e., What are the steps to write assertions for an incomplete unit test when provided with its focal method?) to the LLM model and combined the results to finalize the steps. The first step is to identify the test method and the focal method. Next, developers need to determine the parameters being tested and the assert type, e.g., asserting the parameter value, type/instance, or structural information. The final step involves reasoning the assert value and generating the corresponding statement.

3.3 Query Prompt

Following the submission of the greeting prompt, we introduce the actual query prompt containing the target test cases. The prompt follows a structure similar to the sample code in the greeting prompt but remains incomplete. We also include a brief description in the CoT section to remind the LLM model of the task, generating assertions to replace the assert placeholders, and request to start the CoT process from the first step. The LLM then responds to CLAP and produces the first round of predicted assert statements.

3.4 Feedback Interaction and Prompt

Despite the LLM's ability to generate accurate assert statements, particularly with the CoT prompting which enables it to understand the testing purpose (derived from the test method and focal method names) and locate the parameter to be tested, there are instances where the LLM can not generate the correct value for the target parameters. For example, in situations where the target parameter value relies on complex math calculations (e.g., deep learning models) or is influenced by external factors not immediately evident from the test method or focal method (e.g., parameters defined outside the files or obtained through multiple function calls, self-defined util functions), the LLM may fail to predict the correct value.

To overcome the limitations, we introduce a feedback interaction that allows the LLMs to learn from execution errors in value generation, inspired by prior research [29, 54, 83]. As shown in Fig 1, this process involves executing the initially generated assert statements using the Pytest library [13]. If the assert passed, the LLM has identified the correct value for the tested parameter. In cases where execution fails, CLAP utilises regular expressions to automatically extract key information about the expected and actual values from the error message. CLAP then incorporates the error message into the feedback prompt, ensuring that the LLM is aware of its previous mistake and can use this information to adjust its response generation accordingly for the specific assert placeholder. Fig 3 shows a sample piece of the feedback prompt. Through this feedback interaction, the LLM can progressively generate precise values for target parameters. After all interactions, CLAP collects the second-round predictions and saves them for evaluation.

Project	Revision	# Single Assert Test Method	# Multi-assert Test Method	# Total Test Method	Src
dataclass-json	23e5eec	4	0	4	Pyn
mimesis	978d52c	122	213	335	Pyn
sanic	259e458	169	386	555	Pyn
black	d9b8a64	17	21	38	Pyn & BIP
ansible	367d45f	109	217	326	BIP
cookiecutter	cf81d63	52	97	149	BIP
thefuck	ceaeab	5	6	11	BIP
tornado*	7186b86	299	551	850	BIP
youtube-dl*	0402710	31	89	120	BIP
scrapy*	9cb757d	222	479	701	CLAP
keras*	c72e310	48	106	154	CLAP
detectron2*	d779ea6	41	79	120	CLAP
fairseq*	176cd93	9	33	42	CLAP
mkdocs*	56b235a	67	224	291	CLAP
Total	-	1,195	2,501	3,696	-

Table 1. Characteristics of the Python repositories used in the evaluation.³

4 EVALUATION

To verify the performance of CLAP, we conduct experiments to answer the following research questions:

- **RQ1:** How is the performance of CLAP on LLMs?
- **RQ2:** How do different prompt designs influence prediction performance?
- **RQ3:** How does the CLAP perform in assert generation compare with the state-of-the-art approaches?
- **RQ4:** What are the cases generated by CLAP that failed to accurately match the original test cases?

4.1 Dataset

In our research, we assembled the first dataset for Python assert generation, by examining datasets from the recent popular Python research tools like Pynguin [45], BugsInPy [79], and CODAMOSA [40]. Considering that CODAMOSA utilizes data from Pynguin and BugsInPy, we focused only on these two sources. To avoid the data leakage i.e., testing data being used for model training, we only included project files that were newly added after July 2021, which corresponds to the training data cut-off date for the GPT models [11]. After the removal, we identified 9 projects from the earlier works, with 7 of them using Python assert keyword and 2 employing testing libraries. To increase variety and robustness, we employed the crawling process of ATLAS [75], the first research work in the automatic assert generation field, identifying popular Python projects (stars and forks greater than 30) that also used testing libraries by Github APIs [3]. Combining the 5 most recently updated projects with the initial 9, we created a balanced dataset of 14 projects. These projects are equally distributed between those using the built-in assert keyword and those utilizing testing libraries such as unittest and numpy.testing.

Given a project, CLAP identifies and extracts unit test suites, we wrote automated scripts that could parse the selected repositories, traverse the directory structure, and pinpoint the test files according to the patterns we discussed in Section 3.1. Then, we excluded test methods created before July 2021 to match with the cut-off date. Following the approach used by CODAMOSA, we

employed pipreqs [12] to identify project dependencies and configured the testing environment accordingly. We down-sampled some methods that necessitated additional setup procedures. In total, the dataset consists of 3,696 test methods and 1,298 focal methods (2.85 test methods per focal methods on average). Table 1 provides a detailed overview of the projects included in our dataset.

To match the focal method with the test case, CLAP traverses through the Abstract Syntax Tree (AST) [23] of the unit test code using the *NodeVisitor* class, and identifies the last method call from the same project before or within the assert statements. In Section 2, we discussed two prevalent approaches for writing assert statements. When using the build-in assert, the assert statements consistently begin with the *assert* keyword, while when employing popular testing libraries, the unit test methods generally inherit the test class provided by the libraries, and their assert statements start with *self.assertFunctionName*. In our extraction script, we combine the use of AST and regular expressions to effectively extract assert statements from the original source code. We executed the scripts across all projects and constructed the dataset for evaluation. The dataset shall also benefit relevant studies within the community.

In our testing, the LLM is capable of generating assertions corresponding to the number of placeholders provided. But in the evaluation, we aligned the number of assert placeholders with the original test cases, maintaining a one-to-one correspondence. This process is reflective of typical developer practices when crafting test methods, where developers often write assertions sequentially, having prior knowledge of the required quantity and understanding when the test case is complete [1, 2, 4]. By limiting the number of assert placeholders, we aimed to assess whether CLAP could generate assertions efficiently.

4.2 Evaluation Metrics

We use the following metrics to evaluate the generated assert statements by the LLM.

Accurate Match (AM %). The Accurate Match metric measures the functional equivalence of the generated asserts to the original asserts. In addition to exact matches, we extracted assert statement types from Python testing libraries and identified 34 groups of assert pairs that, while not identical, perform the same function. This concept is inspired by suboptimal test smells [73]. Fig 4 provides some examples of these equivalent assert pairs. For example, in the first pair of assertions, they are functionally same, but the predicted one is preferred as it conveys the intention more explicitly, making the code more readable and maintaining consistency with standard testing practices.

```
def test_net_construct(self, net):
    with self.assertRaisesRegex(ValueError,
                                "output_channels must not be empty"):
        net(output_channels=[],
            kernel_shapes=self.kernel_shapes,
            strides=self.strides,
            paddings=self.paddings)
```

Fig. 4. Example of suboptimal assert match.

LCS (%). We measure the longest common subsequence between the predicted and original assert, calculating the percentage as in Nashid et al. [52], by dividing the subsequence length by the original assert length.

^{3*} denotes the project uses testing libraries. # Test Method is the number of test methods within the project. Src is the project source, where Pyn is for Pynghuin, BIP is for BugsInPy, and CLAP is from this study.

Framework	Assert Num.	Model	AM(%)	LCS(%)	ED	AMM(%)
Python Assert Keyword	Single Assert	text-davinci	64.6	81.1	11.3	-
		gpt-3.5-turbo	55.2	77.1	15.9	-
	Multi Assert	text-davinci	51	74.1	11.7	-
		gpt-3.5-turbo	46.3	69.4	11.5	-
Python Testing Library	Single Assert	text-davinci	66.7	81.8	13.1	84.8
		gpt-3.5-turbo	57.3	76	12.7	77.8
	Multi Assert	text-davinci	68.7	84.4	8.9	83.3
		gpt-3.5-turbo	51	69	10.7	72.1
Average		text-davinci	62	80.6	10.6	83.7
		gpt-3.5-turbo	51	71.1	11.8	73.9

Table 2. Detail statistics of CLAP with *text-davinci* model and *gpt-3.5-turbo* model

Edit Distance (ED). The edit distance is a metric to measure the similarity between two strings by calculating the minimum number of single-character edits (e.g., insertions, deletions, or substitutions) required to transform one into the other. The metric has been widely used to evaluate code-related tasks [27, 31, 52].

Assert Method Match (AMM %). The AMM is a metric to verify whether the predicted assert statement type aligns with the original one. For example, if CLAP predicts an assert statement as *assertIsInstance(a, List)*, but the actual assert statement is *assertIsInstance(a, pd.Array)*, this is still classified as a match under the AMM metric as they both want to check the instance of parameter *a*. The calculation of AMM is represented by the following formula:

$$AMM = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{AssertMethodPredict}_i = \text{AssertMethodOriginal}_i)$$

where $\mathbb{I}(\text{AssertMethodPredict}_i = \text{AssertMethodOriginal}_i)$ is an indicator. When the predicted assert method is equal to the original assert method, $\mathbb{I} = 1$, otherwise, 0.

We use this metric because, the assert statements in Python come in various types, e.g., *assertIn*, *assertIsNone*, *assertAlmostEqual*, and *assertGreaterEqual*. The AMM metric quantifies the precision of CLAP in predicting the appropriate assert method, thereby demonstrating its ability in predicting relevant assert predictions based on the current context. Note that the metric is specifically used to evaluate projects employing testing libraries, as the assert keyword does not distinguish between different assert methods.

4.3 Performance of CLAP (RQ1)

Experiment setup. In this section, we evaluate the prompt design across two LLM models from OpenAI [11]: *text-davinci-003* (*text-davinci*) and *gpt-3.5-turbo*. Inspired by previous works [56] and a small-scale pilot study, we set the temperature parameter to 0.3, striking a balance between allowing some degree of creativity in the model’s responses while ensuring that the predictions remain well-defined. We understand the *code-davinci* is trained on more code repositories, but it was deprecated [11], we have discussed this threat in Section 6. All requests were executed through Python scripts utilizing the official OpenAI Python APIs.

Result. Table 2 shows the results of the assert generation tasks for the two LLMs utilized by CLAP. We can observe that for the Python assert keyword, *text-davinci* outperforms *gpt-3.5-turbo* in both single and multiple assert generation scenarios, achieving 64.6% and 51% accurate match respectively. The prediction on the single assert from *text-davinci* has nearly 10% more accurate assert than the *gpt-3.5-turbo* model. For testing libraries, *text-davinci* still achieves better performance in most metrics. Conversely, *gpt-3.5-turbo* exhibits a lower ED for single assert generation. Overall,

Project	Single Assert				Multiple Asserts				Single + Multi Asserts			
	AM(%)	LCS(%)	ED	AMM(%)	AM(%)	LCS(%)	ED	AMM(%)	AM(%)	LCS(%)	ED	AMM(%)
dataclass-json	100	100	0	-	-	-	-	-	100	100	0	-
mimesis	65.3	79.7	10.5	-	45.2	70.0	11.2	-	51.2	73.3	11	-
sanic	55.6	76	15.9	-	48.7	75.2	10.2	-	50	75.4	11.3	-
black	94.1	95.4	1.3	-	61.5	79.8	6.8	-	81.5	89.5	3.4	-
ansible	71.1	87.1	6.8	-	54.7	72.6	15.0	-	58.5	75.6	13.1	-
cookiecutter	61.5	75.7	15	-	60.4	82.3	14.6	-	60.7	80.5	14.7	-
thefuck	79.2	96.5	2.5	-	50.0	63.0	15.0	-	76.9	93.9	3.4	-
tornado	72.3	84.3	9.2	86.7	62.8	80.3	9.6	79.0	66.2	81.7	9.5	81.7
youtube-dl	83.3	90.1	7.5	91.7	66.7	88.0	7.3	87.5	70.8	88.5	7.3	88.5
scrapy	52.6	78.2	22	82.7	65.6	83.1	10.4	78.6	61.4	82.1	12.7	79.3
keras	59.4	76.8	11.3	71.9	67.3	80.8	11.5	86.7	65.5	79.8	11.4	82.7
detectron2	70.6	84	5.3	91.2	47.1	66.7	14.5	80	48.1	72.4	11.5	83.7
fairseq	60	83.4	4.6	80	55.8	81.5	10.1	92.8	56.2	81.7	9.2	87.5
mkdocs	59.6	79.2	12.9	82.5	83.8	91.5	5.6	91.0	81.7	90.4	6.3	90.3

Table 3. Detail statistics of CLAP per project.

Framework	Assert Num.	CLAP				CLAP_noPersona				CLAP_noCoT				CLAP_noFDBK			
		AM(%)	LCS(%)	ED	AMM(%)	AM(%)	LCS(%)	ED	AMM(%)	AM(%)	LCS(%)	ED	AMM(%)	AM(%)	LCS(%)	ED	AMM(%)
Python Assert	Single Assert	64.6	81.1	11.3	-	62.6	79.9	11.4	-	60.6	78.8	12.1	-	53.8	76.6	13.7	-
Keyword	Multi Assert	51	74.1	11.7	-	47.7	74.2	12.2	-	52.3	76.4	9.9	-	46	73.4	12.1	-
Python Testing	Single Assert	64.7	81.8	13.1	84.8	62.8	81.6	13.9	81.2	62.6	79.7	13.4	78.4	43.5	71.9	19	74
Library	Multi Assert	68.7	84.4	8.9	83.3	60	79.7	11.4	79	67.6	84.3	9	85.6	45.7	71.9	13.8	77.7
Average		62	80.6	10.6	83.7	58.2	78.6	11.5	80	61.5	80.7	10.2	83.7	46.4	73	14.1	77.4

Table 4. Results for prompt design evaluation.

	Equal	True	Is	False	Raises	In	IsInstance	Other
CLAP_noFDBK	48.9	43.8	60	46.8	21.9	26.3	60	31.9
CLAP_Feed	58.7	56.9	65	52.7	30.1	33.3	64.7	37

Table 5. Detailed AM(%) statistics of CLAP for each assert type.

we found the *text-davinci-003* model generally outperforms the *gpt-3.5-turbo* model in generating accurate assert statements. Although *gpt-3.5-turbo* is designed for chat-like interactions, which aligns with our prompt design, it appears to be more cautious and hesitant to avoid making errors. For instance, in some cases, it fails to predict an assert statement because it seeks additional information about method calls beyond the focal methods, particularly for complex test input and multi-assert generation. In contrast, *text-davinci* seems capable of learning about other method calls based on their names and parameters, enabling more accurate predictions. Furthermore, the turbo architecture of *gpt-3.5-turbo* may impact its understanding of longer assertions, resulting in reduced accuracy.

To explore more details, we further analysed the *text-davinci* results at the project level. Table 3 displays the statistics across various projects. *Dataclass-json* achieves a 100% AM, because it only has four assert methods. *Detectron2* has the lowest AM overall, possibly due to its complex DL model structure. *Sanic* and *Scrapy* also show lower AMs, possibly because their web-related nature demands specific testing environments and assert parameters. For 11 out of 14 projects, the AM decreases from the single assert generation to all assert generation. However, *Scrapy*, *Keras*, and *Mkdocs* show an increased AM for all test methods, suggesting higher accuracy in predicting multi-assertions. This can be attributed to these three projects having some test cases with similar structural assert statements, therefore increasing the AM overall.

4.4 Effectiveness of the Prompt Design (RQ2)

Experiment setup. We assess the effectiveness of the prompt design with various input configurations. Existing research has confirmed that providing examples to the LLM can improve accuracy for different tasks [19, 24, 52]. Therefore, we evaluate the other three features within the CLAP prompt design, which include persona, CoT, and feedback prompting. In the experiment, we remove the portions of the prompt that serve these features respectively and maintain all other settings. We then collect the results from different configurations and evaluate their performance.

Result. Table 4 presents the results of the prompt design evaluation. Removing persona design leads to decreased performance, revealing the importance of persona prompts in generating accurate assert statements. The situation is different for the CoT prompt design validation. For single assert statements, CLAP outperforms without CoT for both the Python assert keyword and testing libraries. However, multi-assert generation shows mixed results. For Python assert keyword tests, CLAP_noCoT performs better; for Python testing libraries, CLAP’s prompt achieves higher AM and LCS. Upon examining the multi-assert generation cases responsible for this difference, we identified the following reasons: 1) Some multi-assertion test cases test multiple parameters within a single unit test, which, although common, is considered poor practice for code quality [22]. For instance, a test case might use 8 assertions to test 3 parameters. With the CoT prompt design, the LLM might mistakenly use extra assertions to test the type of parameter A rather than the value of parameter B, ultimately neglecting to test parameter B and resulting in lower AM and AMM. 2) Some test cases are too long and come close to the token limitation of the GPT models, leading the LLM to generate assert statements that might not conform to the desired format. This makes it challenging for CLAP to identify the predictions. Nevertheless, the average AM of CLAP remains higher than the design without CoT. Future work should focus on improving the prompt design for multi-assert statements to address the issues mentioned above.

In general, we observe that the most significant difference across the four metrics lies between CLAP_noFDBK and CLAP, indicating that the feedback interaction design has a substantial impact on performance. Various scenarios may cause the LLM to struggle in predicting the correct value, such as global parameters or functions defined outside the focal method, asserts requiring mathematical calculations, or focal methods with dependencies on other functions. Feedback interaction can assist the LLM in learning from error messages and correcting incorrect predictions. To gain deeper insights into which assert types benefit the most, we provide a breakdown of AM statistics for different assert types in Table 5. The assert types *Is* and *IsInstance* achieve the highest accuracy, possibly due to the complexity of predicting whether objects are the same and determining the object’s type is low. The feedback interaction contributes to the most significant accuracy increase for *Equal* and *True* types, which is because the feedback interaction gives the LLM a better understanding to the expected values. For example, if the *assertEqual* predicted value is incorrect in the first round, the LLM can comprehend the correct value based on the feedback prompt and adjust its prediction accordingly.

Overall, all three prompt components contribute positively to assert generation, with feedback interaction having the most significant impact on performance, while the CoT prompt design needs to be improved for the multi-assert generation.

4.5 Comparison with State-of-the-Art Approaches (RQ3)

Baseline setup. We evaluate the performance of CLAP by comparing it to other state-of-the-art assert generation approaches. ATLAS [75] is the first research work in this field and has demonstrated strong results by adopting a learning-based method. We also fine-tune CODET5 [74], a pre-trained model that is capable of various code-related tasks such as code completion, employing

Approach	AM(%)	LCS(%)	ED	AMM(%)
ATLAS	7.8	38.7	31.3	67
CodeT5	25.1	60	39	80.4
CEDAR	54.7	76.6	16.3	81.5
CLAP	64.7	81.5	12.4	84.8

Table 6. Compare the performance of CLAP with other state-of-the-art approaches.

it as one of the baseline models in our study. CEDAR [52] is another approach that leverages the capabilities of LLMs in combination with information retrieval techniques. Since all prior approaches have exclusively focused on single assert generation, we also evaluate them with the single assert dataset. For ATLAS and CodeT5, we followed their paper instructions to set up the models and validated the result with five-fold cross validation [8]. For CEDAR, we strictly followed their codebase to use the BM25 [59] for the information retrieval and the rest of their code to do the data preprocessing and communicate with OpenAI API. To minimize any potential bias that could be introduced through the IR approach, we adopted the concept of Leave-One-Out [37], which is known to reduce bias while optimizing prediction results in comparison to using a single test set. Specifically, when predicting an assert statement for a given test case, we employ the remaining test cases as the training set for IR, subsequently calculating the evaluation metrics across the entire dataset. Note that since the CODEX model was deprecated, we use the *text-davinci-003* as the LLM to predict. We acknowledge there are other approaches in assertion generations [30, 49, 50]. However, they have some specific settings to Java which presents a challenge to the replication of their method in this study.

Result. Table 6 presents a performance comparison of CLAP with ATLAS, CodeT5, and CEDAR. In all metrics, CLAP outperforms the other approaches. Both CLAP and CEDAR employ LLM to generate assert statements, leading to improvements in AM. Nevertheless, the difference in AMM is relatively small, suggesting that learning-based approach can identify the correct type of assert but fail to accurately capture the specific values or parameters within the assert statement. For example, given a statement *self.assertEqual(out, reqs)*, the output from ATLAS is *self.assertEqual(self, data)*, in which the two parameters within the assert statement do not match any elements from the original one. Sometimes the prediction from ATLAS does not follow the Python syntax, as the learning-based approach relies on patterns observed during training, which may not always adhere to the language's syntax rules. CodeT5 has a similar AMM when compared with CEDAR and CLAP, but falls short in other metrics, particularly ED. We observed instances where CodeT5 failed to terminate its output, resulting in the generation of extraneous and irrelevant text.

CEDAR achieves 54.7% in AM, which demonstrates its capability to generate meaningful assert statements. However, when compared to CLAP, there is still room for improvement. The use of IR includes more shots into the prompt but compared with CLAP, some shots may be inconsistent with the context, affecting the LLM's prediction. For instance, for less common assert types like *assertIsInstance*, CLAP can predict the correct assertions, while CEDAR might be influenced by other samples and predict the value using *assertEqual* or *assertTrue*, which are more widely-used assert types. Besides, the incorporation of CoT and feedback in the prompt design enables CLAP to better predict assert statement values accurately, especially for the complex value structure. For example, given the assert statement *self.assertEqual(jsi.call_function('x'), [20, 20, 30, 40, 50])*, CLAP can identify the correct value with CoT's assistance even before entering the feedback interaction.

Fig 5 shows the distribution of correct assertion lengths for the three approaches. It can be observed that, for shorter assertions, ranging up to 16, CLAP is capable of generating better predictions

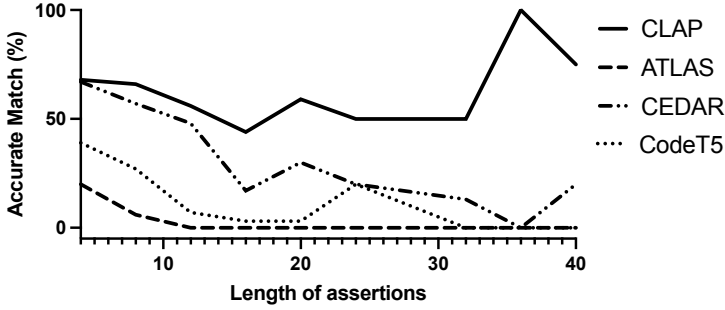


Fig. 5. Length distribution of correct predicted assertions across CLAP, CEDAR, ATLAS, and CodeT5.

in the majority of cases. This can be attributed to the prompt design that assists in comprehending the test input and enables adjustments to the predicted value if it is initially incorrect. For longer assertions, the performance fluctuates, but CLAP still can predict the correct result for long assertions. Nonetheless, CLAP demonstrates improvements in performance and generates more accurate assert statements with less context.

We conducted further examination of the AM assertions generated by CLAP, specifically focusing on cases that are AM but not exact matches. Among the randomly selected 50 test cases, 10% of them are due to the assert parameters are inverted, whereas the rest 90% them are due to readability differences. One prevalent pattern in such cases involves substituting *assertTrue* and *assertFalse* with more descriptive assertions like *assertEqual*, *assertLen*, *assertIsInstance*, and *assertNone* (illustrated in Fig 4). Though functionally identical, the predicted assertions are preferred as they articulate the intention more explicitly, enhancing code readability and aligning with established testing practices, thereby improving the overall quality of the test code. Another pattern pertains to *assertRaises* assertions, where readability can either improve or decline depending on the complexity of the code or developer preference. For instance, in the final example of Fig 4, the predicted assertion is a concise one-line explanation, in contrast to the more space-consuming original code. However, as the lines in the raise block expand, the one-line format commonly produced by CLAP may become more challenging to interpret. The findings highlight CLAP has the ability to increase the readability and maintain consistency with standard testing practices, demonstrating its potential to not only accurately replicate functionality but also enhance the overall quality of the test code.

4.6 Analysis of the Non-AM Assertions (RQ4)

Motivation & Experiment setup.

In this RQ, we conduct a qualitative analysis of the assert statements generated by CLAP that are not AM to the original ones. Through a detailed examination, we can identify the strength or weaknesses of the current approach, and gain valuable insights into the complexities and challenges of automated assert generation. The assessment is conducted by the first two authors who are software engineer PhD students with substantial experience in Python development. We implement an open coding approach [62] to evaluate the generated assertions. A total of 150 test cases are randomly selected from across all non-AM test methods, satisfying a statistically significant sample size that ensures a 95% confidence level and a 0.08 confidence interval [84]. Initially, the first 30 cases are jointly examined by the two authors. Subsequently, we independently label the remaining cases. In instances where the generated asserts are better than the original asserts, we submitted pull requests to the corresponding repositories.

```
def test_classification_metrics(self):
    ...
    expected_return = {'precision': np.array[],
                       'recall': ...}
    metrics = classification_metrics(ground_truth,
                                     retrieved)
    self.assertIsInstance(metrics, dict)
    for k, v in metrics.items():
        self.assertIsInstance(v, np.ndarray)
    ...
```

Fig. 6. Example of some predictions that are non-accurate matches.

Result.

Through the analysis, we identified cases that should be considered as correct predictions, but are challenging to detect using automated scripts. For the assertions that were predicted incorrectly, we also summarized three typical scenarios and investigated the underlying reasons behind these inaccuracies.

22% of the non-AM test cases in our randomly selected dataset should be considered correct predictions. For example, in Fig 6 sample 1, the predicted assertions use the predefined parameter whereas the original one has a hard-coded string. In sample 2, the generated assert checks the response status, whereas the original developer checks the response text. The purpose of both is to check if the response is successful, but they assert different attributes of the HTTP response. The situation varies across different cases, but these examples illustrate that the non-AM test cases may represent alternative but valid ways of performing the assertion as well.

We identified three categories of incorrect predictions in the assertions: value/attribute error (31%), different assert rules (22%), and unrelated (25%). Firstly, the value/attribute error refers to the generated assertions inaccurately reflecting the values being asserted or asserting the wrong attribute of a parameter (sample 3 for example). Though the feedback interaction prompt is designed to help CLAP with getting the correct value, for some complex setups, it may still fail to capture the precise value. Secondly, there are instances where predicted assertions enforce conditions that are either more or less stringent than the original. The fourth sample reveals a stricter predicted assertion, checking the complete equality of the entire dictionaries, as opposed to the original code's specific focus on the `site_name` key. Thirdly, the unrelated cases are where the generated assertions have no clear connection to the real tested parameters, which shows the CLAP may have misapprehended the test case's objective and assessed the wrong parameters. The reasons could stem from the length of the test cases, obscured method calls, or other complexities. In addition, although it's rare for popular open-source projects, we identified a bug in the original test code. In the last sample, the predicted assertion evaluates if date lies within the range of 1 to 31, offering an accurate condition than the original assertion's potentially flawed use of the 'or' operator.

In analyzing non-AM test cases, we found that 22% should be considered correct predictions, representing valid alternatives to the original assertions. Among the incorrect predictions, 53% were closely related to the tested parameters, falling into categories of value/attribute error or different assertion rules. These cases might require minor modifications to align with the original intention. The findings highlight that a significant portion of non-AM cases either align with correct predictions or maintain relevance, underscoring its practical potential.

Following the coding exercise, we conducted a repository-wise summary of the generated assertions that could improve code readability, eliminate test smells, or establish more accurate

Model	Single Assert	Multiple Asserts	Single + Multi Asserts
text-davinci	64.7%	60.7%	62%
gemini-pro	59.1%	53.8%	55.3%
gpt-4.0	67.1%	60.9%	62.5%

Table 7. Accurate match (AM) results across different LLMs.

rules in contrast to the original statements (e.g., sample 5) from both the AM but not exact match cases and the non-AM test cases but should be considered correct predictions. We made submissions in the form of pull requests (PRs) to a total of 9 repositories, resulting in 181 proposed line changes. By the time of this paper’s submission, 7 PRs had been approved or merged while the rest were still under review. We also received thankful comments from project maintainers expressed for our efforts in refining the test code and rectifying potential bugs. These validations from the real software development community highlight the practical utility and efficacy of CLAP in enhancing the quality of unit test code.

5 DISCUSSION

5.1 Generalisation across LLMs

As LLMs gain popularity, various new models are continually being developed by different companies and groups. To evaluate the effectiveness of CLAP’s performance across a range of LLM models, we extended our evaluation beyond the initially tested models. Using the same experimental setup as RQ1, we included additional LLMs, specifically OpenAI’s GPT-4 [18] and Google’s Gemini 1.5 Pro [17]. Table 7 presents the results from these models compared to *text-davinci-003*, which achieved the highest performance in RQ1.

The *gpt-4.0* achieves higher AM across all fields, likely due to being the latest model from OpenAI. The extended token limits enables it to handle longer test cases and more multiple assertions, which might be challenging for *text-davinci*. A new finding from the non-AM predictions of *gpt-4.0* is it tends to include additional conditions compared to the original ones. For example, where the original assertion is *assert A*, *gpt-4.0* might predict *assert A and B*, with *B* being a valid condition, but not originally specified by the developer.

The performance of *gemini-pro* is not as good as the two OpenAI models but still outperforms state-of-the-art approaches for single assert prediction (in RQ3). We observed that *gemini-pro* sometimes failed to respond in the requested format, leading to parsing errors and stopped before the feedback interaction phase due to these errors. To enhance the performance of *gemini-pro*, further refinements in prompt design or a more flexible parser for LLM responses may be necessary.

In summary, the results demonstrate that the design of CLAP works well with different models, highlighting its generalisation across different LLMs. Depending on the LLMs, refinements may be necessary for specific models. These refinements might involve minor adjustments in prompt design or response parsing strategies, but do not require major changes to its core structure.

5.2 Practical Implications of CLAP

In this section, we discuss two practical usage of CLAP: (1) as a wrapper for LLMs, integrated within an IDE or CI process, and (2) to enhance the assert generation capabilities of existing Python unit test generation tools.

Usage Scenarios for Developers CLAP can be used as an assistant for developers to help them complete the test cases in their projects. For example, John is a developer for an opensource project. He has just written a new function and is in the process of writing a unit test for the corresponding function. He writes the test inputs but is unsure about the specific assertions needed to thoroughly

test the new function. CLAP can work as a plugin to his IDE. It automatically analyse the code context and suggest the correct assert statements for John.

CLAP can also be used in continuous integration (CI) environments. When new test code is committed to a repository, CLAP can automatically analyse the test code. In cases where the test code may contain bugs or smells (e.g., tests that always pass regardless of the code behavior⁴), CLAP can compare the generated assert statements with those written by developers. If discrepancies are found, CLAP can notify the developers, providing both versions of assert statements and prompting them to decide which one is more appropriate. Additionally, if the new test code fails, CLAP can analyse the code context and suggest alternative assert statements using its feedback interaction mechanism. So if the issue lies within the original assert statements, developers can quickly identify the issue and replace the asserts.

Integration with Automatic Unit Test Generation Tool Pynguin [45, 46] is a research framework that can automatically generate unit tests for Python and its Github repository has received over 1.1k stars. Due to its design, it requires the target projects to have type annotations. Two of the projects in our dataset overlap with those used in their research, namely Mimesis and Sanic. We employed Pynguin version 0.33.0 with the configuration “*pynguin -algorithm DYNAMOSA -assertion_generation MUTATION_ANALYSIS*” to generate unit tests for these codebases. We then removed the test cases which were tagged with *pytest.mark.xfail* decorator. These are the cases where Pynguin catch an unexpected error while running the test case. After manual inspect, we found this is due to failed to feed the correct typed parameter of a function. Then, for test cases that failed to produce assertions or the assertions failed, we executed CLAP to generate the necessary assertions and subsequently compared the resulting code coverage.

In total, Pynguin generated 98 test cases without the xfail docstring for Sanic and Mimesis. Among them, 25 missed an assert statement. For example, the Pynguin generated cases could have assigned a parameter from the focal method without asserting the parameter. After execution of CLAP, we successfully generated assertions for the untested parameters in all the missing cases and fixed 2 failed test cases where Pynguin generated the wrong assertions. And result in an increased 2% test coverage for Sanic and Mimesis. These results indicate that CLAP can complement Pynguin by rectifying its limitations and enhancing the quality of the automatically generated test cases. Further experiments are necessary to explore how Pynguin can better utilize CLAP’s capabilities, perhaps by integrating CLAP’s functionality into its asserts generation process.

Moreover, prior studies such as CODAMOSA [40] have also explored the potential of LLM models to generate meaningful inputs for the test cases generated from Pynguin, with the intention of mitigating the incidence of test cases decorated with *pytest.mark.xfail* and thus increased the code coverage. By integrating these efforts, there exists prospective future research to enhance the quality of automatically generated unit tests. This could involve the development of a more comprehensive framework that capitalizes on the strengths of LLMs to not only create more meaningful assert statements but also generate the entire test case. It could potentially lead to a improvement in auto-generated unit tests, making them more reliable and beneficial for Python projects.

5.3 Multiple Asserts and Assert Roulette

Test smells, similar to code smell, are known for bad design or implementation decisions in the unit test code [20, 70, 73]. Over the years, different test smells in languages such as Java, Javascript, Python were raised by researchers. One of the earliest and most prevalent of these is Assert Roulette [70], characterized by test cases that contain multiple assert statements without explanatory messages or documentation [73]. It can be harmful as developers need more efforts to figure out

⁴<https://github.com/lk-geimfari/mimesis/pull/1385>

why the test case failed. It has been one of the most common test smells among languages [73]. Our randomly sampled data set reveals that 71.8% of the test cases include more than one assert statement.

However, recent research by Panichella et al. [55] suggests that the concept of Assertion Roulette may need updating due to advancements in testing frameworks. For example, the unittest framework in Python provides detailed feedback about which asserts fail and the reasons for the failure (e.g., actual vs. expected value). The message is clear enough for the developers to identify the issues. Furthermore, the choice between using multiple assertions in a single test and spreading them across multiple tests often comes down to a trade-off between assertion roulette and code duplication. Developers often prefer multiple assertions within a single test when these assertions are strongly related, as this approach reduces redundancy. For example, in the Scrapy project, there is a `test_send_html` function⁵, two assertions check that both the body and the content type of the sent HTML meet expectations. It is more efficient and clearer to test these aspects together in a single test case rather than duplicating the setup in separate tests.

Although Assert Roulette is traditionally considered a test smell, it no longer negatively impacts unit tests as it once did, with the evolving testing frameworks and the way developers write unit tests. Therefore, CLAP's ability to generate multiple assertions within a single test case remains a significant and valuable contribution.

6 THREATS TO VALIDITY

Internal validity. A potential threat to internal validity in our study is the extraction of focal methods. We adopted a heuristic approach from previous research [75] to match focal methods with test cases. However, since Python test cases can be structured differently compared to Java, biases may be introduced in the identified focal methods. To address this issue, we modified the script to exclude cases where the focal method was unclear. Additionally, in real-world scenarios, we expect users to be developers who can provide the correct corresponding focal method. Despite this limitation, our approach demonstrates its effectiveness in generating meaningful assert statements, reflecting its robustness.

External validity. Our study focuses on generating assert statements for Python projects that present potential limitations in external validity, specifically concerning the generalizability of our approach. While the LLMs utilized in our approach have demonstrated versatility across various languages (e.g., CEDAR [52]), we did not conduct an empirical evaluation on other widely used datasets, such as the Java dataset from ATLAS. This was mainly due to the need for executing real projects, which did not align with the ATLAS database's constraints. Although our method shows the capability to address dynamically-typed languages like Python that possess a greater diversity of assertion types and complex type check compared with Java, the generalization to statically-typed languages remains untested. However, because of the use of the LLM, CLAP can potentially be generalized across various programming languages with minor adjustments to the interpreter environment for feedback interaction and the prompt manager design. This indicates a promising scope for CLAP's widespread applicability in the assert generation task for a diverse range of programming languages.

While our dataset represents the first dedicated collection for Python assert generation, its 14 projects constructed dataset presents a limitation in generalisability. Despite our efforts to ensure diversity by including projects from various domains (Python libraries, ML/DL repositories, command line tools, and applications) and balancing those using built-in assert keywords with testing libraries, the relatively small number of projects may not capture all Python testing practices.

⁵https://github.com/scrapy/scrapy/blob/10a843ac1d7105d1503ab367e3113da20a2d36fc/tests/test_mail.py#L46

This constraint is due to the large effort required to configure and prepare each project for analysis, and the necessity to avoid data leakage for the LLMs. Future works should aim to expand this dataset to further enhance generalisability of the findings.

Another potential threat is the adaptability of our prompt design to a variety of LLMs. Given the current popularity of the LLM field, new models are being introduced frequently and old ones may be deprecated (e.g., CODEX). In the experiment and discussion section, we evaluate our approach with four LLMs, and the results demonstrate that all models can generate meaningful assert statements using the current prompt design while a minor modification will be needed for different LLMs. As LLM development continues to evolve, there may be differences in input requirements; however, the core concept of providing natural language prompts to LLMs is expected to remain consistent. Therefore, only limited efforts would be required to adapt our approach to different LLM models.

Construct validity. Throughout the study, our aim has been to generate meaningful assert statements; however, the term “meaningful” lacks a clear definition. In line with previous studies [30, 49, 52, 75, 82], we evaluate the results by comparing them with the original asserts from developers. However, it is important to acknowledge that these original asserts may have their own limitations, such as poor readability, ineffectiveness in bug detection, or the presence of test smells. There are other metrics such as the checked coverage [61] that can evaluate the quality of the generated test code dynamically rather than the static string-based metrics. However, as Schafer et al. [60] pointed out, for dynamically typed language (e.g., JS and Python), it is difficult to precisely implement and calculate the checked coverage. This could potentially affect the construct validity of our study. We recognise this approach to be an assistant to help developers write assert statements and conduct RQ4 to analyse the differences between the generated asserts and the original ones. Future works could focus on involving a more comprehensive evaluation process and improving the generated assert code quality.

7 RELATED WORK

7.1 Assert Generation

In 2020, Watson et al. [75] first introduced a Deep Learning (DL) method for generating assert statements in Java, referred to as ATLAS. Utilizing a Neural Machine Translation (NMT) based approach, their work aimed to generate meaningful assert statements, which had proven to be a challenge for automatic unit test generation. Moreover, they extracted test cases from Java projects, paired them with their corresponding focal methods, and termed this structure as Test-Assert Pairs (TAPs). They released their dataset for the benefit of future research in the field.

Subsequently, Yu et al. [82] extended ATLAS by integrating an Information Retrieval (IR) based approach, while Tufano et al. [69] fine-tuned a state-of-the-art transformer model [41], both achieving better performance compared to ATLAS. Similarly, Mastropaolo et al. [49, 50] trained T5 model with large Java Dataset and fine-tuned with assert code, also has achieved better performance in assert generation. Most recently, Nashid et al. [52] combined IR with LLM to generate asserts in Java, achieving state-of-the-art performance. Their tool, CEDAR, which implements the BM25 algorithm to find the most similar test cases from a pre-built dataset and fed into CODEX (an LLM model from OpenAI) as few-shot learning prompts to generate the assert statements. In contrast to previous works, which focused on generating single asserts for Java projects, our work addresses the currently most popular language, Python, which has not been previously explored in this context and tends to have more multi-assert test cases. Furthermore, all three previous works relied on pre-built datasets to support their performance, potentially introducing bias and limitations in generating assert statements. Other than Java, Zamprognio et al. [83] introduced AutoAssert,

a system designed to generate assertions for JavaScript. To address the limitations of dynamic approaches in assertion generation, they implemented a human-in-the-loop mechanism by allowing developers to choose the parameters to be tested and select suitable assertions from a predefined set. Like our method, AutoAssert assumes the tested code is correct, but its support is restricted to a limited number of assertion types due to inherent limitations in its design.

7.2 Large Language Models for Software Engineering

Recently, various LLMs have been produced by different organizations, such as RoBERTa [44], GPT models [11], PaLM [51], LLAMA [66, 67]. In the software engineering domain, researchers and practitioners have conducted numerous experiments to explore the potential contributions of LLMs to the community. Copilot [35] assists developers in code writing by reading the code context and comments to generate code suggestions. Codium.AI [7] utilized LLM to generate unit tests for developers and developed plugins for the IDEs. CODAMOSA [40] leveraged OpenAI's CODEX to provide example test cases to help with unit test generation. Xia et al. [80] and Nashid et al. [52] discussed how the LLM can help with program repair to fix potential bugs. Wang et al. [72] demonstrated that LLMs exhibit a deeper understanding of source code in tasks like algorithm classification, code clone detection, and code search. And Khan et al. [38] used LLM to generate meaningful documents for different programming languages. Our project aims to capitalize on the LLM's understanding of source code and incorporate the latest prompt engineering methods to assist developers in generating meaningful assert statements for Python projects.

8 CONCLUSION AND FUTURE WORK

In this study, we present CLAP, an approach that integrates rounds of prompt communication with persona design, CoT prompting, and feedback interactions to optimize the performance of LLMs in generating meaningful assert statements for Python. Our evaluation demonstrates that CLAP outperforms the state-of-the-art approach in producing accurate single assert statements by over 10%. Adding consideration of multi-asserts generation, which is more common in Python projects, CLAP is also capable to generate accurate match assertions at 62%. We further conducted a qualitative study that suggested the generated assertions have the potential to increase the readability of the test case and fix the unseen bugs within the original assertions. Our submitted pull requests based on the generated assertions that positively influenced the test cases were accepted by real open-source projects. In addition, we discuss the generalisation across LLMs, practical applications of CLAP, and the importance of multi-asserts generation. In the future, we plan to further refine the method to improve and assess the quality of generated assert statements and minimize test smells. Given the compliant nature of LLMs like the GPT models, we will investigate more robust strategies to ensure these systems not only accept corrections but also apply them in a way that demonstrates a deeper understanding of code exceptions and errors in the prompt engineering aspect. From the practical perspective, we intend to develop this tool into a plugin for Python IDEs to simplify access for developers. Ultimately, we would like to extend the work to automatic unit test generation to better benefit the software engineer community.

REFERENCES

- [1] 2012. Arrange Act Assert. <http://wiki.c2.com/?ArrangeActAssert>. Accessed: 2023-07-29.
- [2] 2020. One assertion per test? <https://pierodibello.medium.com/one-assertion-per-test-732cc2a7d3d>. Accessed: 2023-07-29.
- [3] 2022. GitHub REST API. <https://docs.github.com/en/rest>. Accessed: 2023-04-09.
- [4] 2022. Write Professional Unit Tests in Python. <https://code.tutsplus.com/write-professional-unit-tests-in-python--cms-25835t>. Accessed: 2023-07-29.
- [5] 2023. awesome-chatgpt-prompts. <https://github.com/f/awesome-chatgpt-prompts>. Accessed: 2023-03-29.

- [6] 2023. CLAP Github Repository. <https://github.com/freddiewanah/CLAP>. Accessed: 2023-10-06.
- [7] 2023. Codium AI Generating meaningful tests for busy devs. <https://www.codium.ai/>. Accessed: 2023-04-03.
- [8] 2023. A Gentle Introduction to k-fold Cross-Validation. <https://machinelearningmastery.com/k-fold-cross-validation/>. Accessed: 2023-05-01.
- [9] 2023. How to Write Unit Tests. <https://www.wikihow.com/Write-Unit-Tests>. Accessed: 2023-07-03.
- [10] 2023. Module: tf.test Tensorflow. https://www.tensorflow.org/api_docs/python/tf/test. Accessed: 2023-07-03.
- [11] 2023. OpenAI Models. <https://platform.openai.com/docs/models/overview>. Accessed: 2023-07-03.
- [12] 2023. pipreqs · PyPI. <https://pypi.org/project/pipreqs/>. Accessed: 2023-07-03.
- [13] 2023. pytest: helps you write better programs. <https://docs.pytest.org/en/7.3.x/>. Accessed: 2023-07-03.
- [14] 2023. scikit-learn Machine Learning in Python. <https://scikit-learn.org/stable/index.html>. Accessed: 2023-07-03.
- [15] 2023. TIOBE Index for March 2023. <https://www.tiobe.com/tiobe-index/>. Accessed: 2023-03-29.
- [16] 2023. unittest — Unit testing framework. <https://docs.python.org/3/library/unittest.html>. Accessed: 2023-07-03.
- [17] 2024. Gemini. <https://ai.google.dev/gemini-docs/models/gemini>. Accessed: 2024-05-29.
- [18] 2024. OpenAI Models. <https://platform.openai.com/docs/models>. Accessed: 2024-05-29.
- [19] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *37th IEEE/ACM International Conference on Automated Software Engineering*. 1–5.
- [20] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian D Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test smell detection tools: A systematic mapping study. In *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*. 170–180.
- [21] M Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 263–272.
- [22] Mauricio Finavaro Aniche, Gustavo Ansaldo Oliva, and Marco Aurélio Gerosa. 2013. What do the asserts in a unit test tell us about code quality? a study on open source and industrial projects. In *2013 17th European Conference on Software Maintenance and Reengineering*. IEEE, 111–120.
- [23] Ira D Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant’Anna, and Lorraine Bier. 1998. Clone detection using abstract syntax trees. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE, 368–377.
- [24] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [25] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-chi Cheung. 2023. A study on Prompt Design, Advantages and Limitations of ChatGPT for Deep Learning Program Repair. *arXiv preprint arXiv:2304.08191* (2023).
- [26] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2023. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Transactions on Software Engineering* (2023).
- [27] Saikat Chakraborty and Baishakhi Ray. 2021. On multi-modal learning of editing source code. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 443–455.
- [28] Ermira Daka and Gordon Fraser. 2014. A survey on unit testing practices and problems. In *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE, 201–211.
- [29] Brett Daniel, Vilas Jagannath, Danny Dig, and Darko Marinov. 2009. ReAssert: Suggesting repairs for broken unit tests. In *2009 IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 433–444.
- [30] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
- [31] Yangruibo Ding, Baishakhi Ray, Premkumar Devanbu, and Vincent J Hellendoorn. 2020. Patching as translation: the data and the metaphor. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 275–286.
- [32] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.
- [33] Davide Fucci and Burak Turhan. 2014. On the role of tests in test-driven development: a differentiated and partial replication. *Empirical Software Engineering* 19 (2014), 277–302.
- [34] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2023. An Empirical Study on Using Large Language Models for Multi-Intent Comment Generation. *arXiv preprint arXiv:2304.11384* (2023).
- [35] GitHub. 2022. GitHub Copilot · Your AI pair programmer. <https://github.com/features/copilot>. Accessed: 2023-04-03.

- [36] Danielle Gonzalez, Joanna CS Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. 2017. A large-scale study on the usage of testing patterns that address maintainability attributes: patterns for ease of modification, diagnoses, and comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 391–401.
- [37] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. 2009. *The elements of statistical learning: data mining, inference, and prediction*. Vol. 2. Springer.
- [38] Junaed Younus Khan and Gias Uddin. 2022. Automatic Code Documentation Generation Using GPT-3. In *37th IEEE/ACM International Conference on Automated Software Engineering*. 1–6.
- [39] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916* (2022).
- [40] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering, ser. ICSE*.
- [41] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
- [42] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A Sketch-based Approach for Automatic Code Generation. *arXiv preprint arXiv:2302.06144* (2023).
- [43] Vivian Liu and Lydia B Chilton. 2022. Design guidelines for prompt engineering text-to-image generative models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–23.
- [44] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [45] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 168–172.
- [46] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. *Empirical Software Engineering* 28, 2 (2023), 36.
- [47] David R MacIver and Alastair F Donaldson. 2020. Test-case reduction via test-case generation: Insights from the hypothesis reducer (tool insights paper). In *34th European Conference on Object-Oriented Programming (ECOOP 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [48] Paula Maddigan and Teo Susnjak. 2023. Chat2vis: Generating data visualisations via natural language using chatgpt, codex and gpt-3 large language models. *arXiv preprint arXiv:2302.02094* (2023).
- [49] Antonio Mastropaolo, Nathan Cooper, David Nader Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2022. Using transfer learning for code-related tasks. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1580–1598.
- [50] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 336–347.
- [51] Sharan Narang and Aakanksha Chowdhery. 2022. Pathways Language Model (PaLM): Scaling to 540 Billion Parameters for Breakthrough Performance.
- [52] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE’23)*.
- [53] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot’s code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 1–5.
- [54] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. *arXiv preprint arXiv:2302.08468* (2023).
- [55] Annibale Panichella, Sebastiano Panichella, Gordon Fraser, Anand Ashok Sawant, and Vincent J Hellendoorn. 2022. Test smells 20 years later: detectability, validity, and reliability. *Empirical Software Engineering* 27, 7 (2022), 170.
- [56] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. ART: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014* (2023).
- [57] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI’s codex fix bugs? an evaluation on QuixBugs. In *Proceedings of the Third International Workshop on Automated Program Repair*. 69–75.
- [58] Abdallah Qusef, Rocco Oliveto, and Andrea De Lucia. 2010. Recovering traceability links between unit tests and classes under test: An improved method. In *2010 IEEE International Conference on Software Maintenance*. IEEE, 1–10.
- [59] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

- [60] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* (2023).
- [61] David Schuler and Andreas Zeller. 2011. Assessing oracle quality with checked coverage. In *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 90–99.
- [62] Carolyn B. Seaman. 1999. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on software engineering* 25, 4 (1999), 557–572.
- [63] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 201–211.
- [64] Kunal Taneja and Tao Xie. 2008. DiffGen: Automated regression unit-test generation. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 407–410.
- [65] Ayse Tosun, Muzamil Ahmed, Burak Turhan, and Natalia Juristo. 2018. On the effectiveness of unit tests in test-driven development. In *Proceedings of the 2018 International Conference on Software and System Process*. 113–122.
- [66] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [67] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [68] Fabian Trautsch and Jens Grabowski. 2017. Are there any unit tests? an empirical study on unit testing in open source python projects. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 207–218.
- [69] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*. 54–64.
- [70] Arie Van Deursen, Leon Moonen, Alex Van Den Bergh, and Gerard Kok. 2001. Refactoring test code. In *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. Citeseer, 92–95.
- [71] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [72] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging pre-trained models and downstream tasks for source code understanding. In *Proceedings of the 44th International Conference on Software Engineering*. 287–298.
- [73] Tongjie Wang, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, and Iftekhar Ahmed. 2021. Pynose: a test smell detector for python. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 593–605.
- [74] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
- [75] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1398–1409.
- [76] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903* (2022).
- [77] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
- [78] James A Whittaker. 2000. What is software testing? And why is it so hard? *IEEE software* 17, 1 (2000), 70–79.
- [79] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, et al. 2020. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1556–1560.
- [80] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the ACM/IEEE 45th International Conference on Software Engineering (ICSE’23)*.
- [81] Hiroaki Yoshida, Susumu Tokumoto, Mukul R Prasad, Indradeep Ghosh, and Tadahiro Uehara. 2016. FSX: fine-grained incremental unit test generation for C/C++ programs. In *Proceedings of the 25th international symposium on software testing and analysis*. 106–117.

- [82] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated assertion generation via information retrieval and its integration with deep learning. In *Proceedings of the 44th International Conference on Software Engineering*. 163–174.
- [83] Lucas Zamprogno, Braxton Hall, Reid Holmes, and Joanne M Atlee. 2022. Dynamic Human-in-the-Loop Assertion Generation. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2337–2351.
- [84] Jerrold H Zar. 1999. *Biostatistical analysis*. Pearson Education India.
- [85] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493* (2022).